

1.

org 100h

```
    ; --- Código Principal ---
    lea dx, msg      ; Carrega o endereço da mensagem em DX
    call print_str   ; Chama a procedure

    ret              ; Encerra o programa aqui. O processador para.

; --- Procedure ---
print_str:
    mov ah, 09h
    int 21h
    ret

; --- Dados ---
; Como o 'ret' lá em cima já parou o programa,
; o processador nunca vai tentar "executar" essa linha abaixo.
msg db 'Ola, mundo!$'
```

2.

org 100h

```
    ; 1. Ler um caractere do teclado
    mov ah, 01h      ; Função 01h: Lê um caractere e guarda em AL
    int 21h          ; Executa a leitura (o programa pausa aqui até você digitar)

    ; --- O caractere digitado agora está no registrador AL ---

    ; 2. Preparar para imprimir
    ; A sua procedure print_char usa a função 02h.
    ; A função 02h exige que o caractere esteja em DL.
    ; Precisamos copiar o que está em AL para DL.
    mov dl, al       ; Copia o caractere lido para o lugar de impressão

    ; 3. Imprimir (Ecoar)
    call print_char  ; Chama a procedure para mostrar o que está em DL

    ret              ; Encerra o programa

; --- Procedure Solicitada ---
print_char:
    mov ah, 02h      ; Função 02h: Imprime caractere
    int 21h
    ret
```

3.

org 100h

```
    ; Inicializa DL com o primeiro caractere ('1')
    mov dl, '1'
    ; Imprime '1'
```

```

call print_char

; Incrementa DL (de '1' vira '2') e imprime
inc dl
call print_char

; Incrementa DL (vira '3') e imprime
inc dl
call print_char

; 4
inc dl
call print_char

; 5
inc dl
call print_char

; 6
inc dl
call print_char

; 7
inc dl
call print_char

; 8
inc dl
call print_char

; 9
inc dl
call print_char

ret ; Encerra o programa

; --- Procedure ---
print_char:
    mov ah, 02h
    int 21h
    ret

4.
org 100h

; 1. Preparar o contador
mov cx, 10          ; Define que o loop vai rodar 10 vezes

; 2. Preparar o caractere (Isso não muda, então pode ficar fora do loop)
mov dl, '*'

inicio_loop:
; --- O que acontece DENTRO do loop ---

```

```

; Chama a função de imprimir
mov ah, 02h
int 21h

; --- O comando mágico LOOP ---
loop inicio_loop

; Quando o loop terminar (CX chegar a 0), o código continua aqui
ret

```

5.

org 100h

```

; 1. Carregar e Somar
mov al, a      ; AL = 5
add al, b      ; AL = 5 + 7 = 12
mov c, al      ; Armazena 12 na variável c

; 2. Preparar para exibir (Separar dezenas e unidades)
; Vamos limpar AH para usar a divisão
xor ah, ah     ; AH = 0, então AX = 0012 (decimal)
mov bl, 10
div bl         ; AX / 10 -> AL = Resultado (1), AH = Resto (2)

; Agora: AL tem 1, AH tem 2. Vamos salvar o resto (2) para depois
mov dx, ax     ; Salvo cópia em DX (DL=1, DH=2) para não perder

; 3. Imprimir a Dezena (que está em DL)
add dl, 30h    ; Converte número 1 para caractere '1' (ASCII 49)
call print_char

; 4. Imprimir a Unidade (que estava em DH)
mov dl, dh     ; Recupera o resto (2) para DL
add dl, 30h    ; Converte número 2 para caractere '2' (ASCII 50)
call print_char

ret           ; Fim

```

; --- Procedure de Impressão ---

```

print_char:
    mov ah, 02h
    int 21h
    ret

```

; --- Dados ---

```

a db 5
b db 7
c db ?

```

6.

org 100h

```

; Vamos simular que AL tem um valor numérico entre 0 e 9

```

```

mov al, 5          ; Exemplo: valor numérico 5

; --- Resolução do exercício ---
add al, 30h        ; 5 + 48 = 53 (Que é o código ASCII do caractere '5')

; Exibir para provar que funcionou
mov dl, al
mov ah, 02h
int 21h

ret

```

7.
org 100h

```

mov ah, 01h
int 21h            ; AL agora tem o caractere (ex: '5' -> ASCII 53)
sub al, 30h        ; Converte ASCII '5' (53) para número 5
mov bl, al         ; Guarda o 5 em BL

mov ah, 01h
int 21h            ; AL tem o caractere (ex: '7' -> ASCII 55)
sub al, 30h        ; Converte '7' para número 7

add al, bl         ; AL = 7 + 5 = 12

mov bl, al
mov dl, 10
mov ah, 02h
int 21h
mov al, bl

xor ah, ah
mov cl, 10
div cl

mov dx, ax

cmp dl, 0
je imprime_unidade

add dl, 30h
mov ah, 02h
int 21h
imprime_unidade:

```

```
    mov dl, dh
    add dl, 30h
    mov ah, 02h
    int 21h
```

```
    ret
```

8.

```
org 100h
```

```
    lea si, buffer
```

```
ler_loop:
```

```
    mov ah, 01h
    int 21h
```

```
    cmp al, 0Dh
    je fim_leitura
```

```
    mov [si], al
    inc si
    jmp ler_loop
```

```
fim_leitura:
```

```
    mov [si], '$'
```

```
    call nova_linha
```

```
    lea dx, buffer
    mov ah, 09h
    int 21h
```

```
    ret
```

```
nova_linha:
```

```
    mov ah, 02h
    mov dl, 10      ; Line Feed
    int 21h
    mov dl, 13      ; Carriage Return
    int 21h
    ret
```

```
buffer db 50 dup(?)
```

9.

```

org 100h

    lea si, texto
    mov bl, 0

verificar_char:
    mov al, [si]

    cmp al, '$'
    je fim_contagem

    cmp al, 'A'
    jne proximo

    inc bl

proximo:
    inc si
    jmp verificar_char

fim_contagem:

    add bl, 30h
    mov dl, bl
    mov ah, 02h
    int 21h

    ret

texto db 'ANA BANANA BACANA$'

```

10.

```

org 100h

    mov cx, 5
    dec cx

loop_externo:
    mov bx, cx
    lea si, vetor

loop_interno:
    mov al, [si]
    mov ah, [si+1]

    cmp al, ah
    jbe nao_troca

    mov [si], ah
    mov [si+1], al

```

```
nao_troca:
    inc si
    dec bx
    jnz loop_interno

    loop loop_externo


    lea si, vetor
    mov cx, 5

imprimir_vetor:
    mov dl, [si]
    add dl, 30h
    mov ah, 02h
    int 21h

    mov dl, ' '
    int 21h

    inc si
    loop imprimir_vetor

    ret
```

```
vetor db 5, 2, 9, 1, 6
```